WaitNotifyExample:-> ProducerConsumerSystem:->

```
Order.java:->
```

```java
package com.example.demo;

public record Order(int orderId,String productName,int quantity) {}
```

```
SharedBuffer.java:->
```

```java
package com.example.demo;

import java.nio.Buffer;
import java.util.LinkedList;
import java.util.List;

public class SharedBuffer {
    private final List<Order>buffer=new LinkedList<>();
    private final int capacity;

    public SharedBuffer(int capacity)
    {
        this.capacity=capacity;
    }

    public synchronized void produce(Order order) throws InterruptedException
    {
        while (buffer.size()==capacity)
        {
            System.out.println("Buffer is full. Producer is waiting...");
            wait();
        }
        buffer.add(order);
        System.out.println("New order added: "+order);
        notifyAll();
    }

    public synchronized  Order consume() throws InterruptedException{
        while (buffer.isEmpty())
        {
            System.out.println("Buffer is empty.Consumer is waiting...");
            wait();
        }
        Order order=buffer.removeFirst();
        System.out.println("Processing Order:"+order);
        notifyAll();
        return order;


    }

}
```

Producer.java:->

```java
package com.example.demo;

import java.util.Random;

public class Producer implements Runnable{
private final SharedBuffer sharedBuffer;
private int orderId=0;

public Producer(SharedBuffer sharedBuffer)
{
    this.sharedBuffer=sharedBuffer;
}
    @Override
    public void run() {
        String[] products = {"Laptop", "Phone", "Headphone", "Monitor",
"Keyboard"};

        Random random=new Random();
        while (true)
        {
            try{
                String productName=products[random.nextInt(products.length)];
                int quantity=random.nextInt(1,6);

                Order order=new Order(++orderId,productName,quantity);
                sharedBuffer.produce(order);
                Thread.sleep(1000L);

            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                break;
            }
        }

    }
}
```

Consumer.java:->

```java
package com.example.demo;

public class Consumer implements Runnable{
    private final SharedBuffer sharedBuffer;

    public Consumer(SharedBuffer sharedBuffer)
    {
        this.sharedBuffer=sharedBuffer;
    }
    @Override
    public void run() {
        while (true)
        {
            try{
                Order order=sharedBuffer.consume();
                System.out.println("Delivered order:"+order);
                Thread.sleep(2000L);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                break;
            }
        }

    }
}
```

MainTest.java:->

```java
package com.example.demo;

public class MainTest {
    public static void main(String[] args) {
        SharedBuffer sharedBuffer=new SharedBuffer(5);
        Thread producerThread=new Thread(new Producer(sharedBuffer));
        Thread consumerThread=new Thread(new Consumer(sharedBuffer));
        producerThread.start();
        consumerThread.start();

    }
}
```